Gated Depth-Readers Lose to Deeper Trunks: a per-FLOP study at d=10

2d-Transformers

June 25, 2026

1 Motivation

A depth-L transformer builds a *residual stream*: starting from the token embedding x_0, each layer adds its update, producing a ladder of hidden states

$$\Lambda \;=\; [\,x_0,\; h_1,\; h_2,\; \ldots,\; h_L\,] \qquad (L{+}1 \text{ “rungs”}).$$

A standard model reads out only the **top rung** h_L through the language-model head. But the intermediate rungs hold features at different levels of abstraction, and h_L is a single-vector bottleneck. This raises a natural question:

> *Can a model produce a better readout by reading its **entire depth** Λ, rather than just the last layer?*

Six earlier experiments tested a *depth-reader* that **replaces** h_L (it “owns” the output). It never beat the plain top-state baseline. Replacing a good representation may be too aggressive — so here we test the gentlest possible version: the reader as a small, **gated additive correction** to h_L, which at initialization changes nothing and must *earn* its influence.

2 Architecture: the gated depth-reader

On top of the usual backbone H (depth L, width e) we add a small reader V:

- **Reader V.** A small transformer of V blocks at full width e (same block shape as the backbone). It attends over the *depth* axis — the $L{+}1$ rungs of Λ are treated as a sequence — and returns a correction vector $r = V(\Lambda)$.
- **Gated readout.** Instead of $x = h_L$, the model uses

$$\boxed{\text{readout} \;=\; \underbrace{h_L}_{\text{baseline top state}} \;+\; g \cdot V(\Lambda)} \qquad g \in \mathbb{R}, \quad g_0 = 10^{-3}.$$

- **The gate** g is a single scalar, initialized ≈ 0, with its own optimizer group, logged at every evaluation.

The design makes the comparison honest. At g=0 the model is *exactly* the baseline, so the reader can never start out worse. The tiny g_0 still gives V a gradient from step 0, so it learns immediately. If reading the ladder helps, g grows and we see it; if it does not, g stays near zero. “Did the reader turn on?” becomes a *measured* quantity, not an assumption.

3 Experiment: the V-slope at d=10

We fix the backbone at d=10 (e=640) and sweep the reader depth $V \in \{2, 4, 8, 10\}$, all gated. Every model is trained compute-optimally (tokens = 12× params). A reader is a *bigger* model (H+V parameters), so it is fairly given proportionally more tokens.

The decisive question is **per-FLOP**: for the same total compute, you could instead train a *deeper plain transformer*. So we also train plain baselines d=12, 14, 16 (and reuse d=10), giving a loss-versus-compute **frontier**. The verdict for each reader is simply: does it land *below* that frontier (a per-compute win) or *above* it (a loss)? Quality is validation bits-per-byte (BPB, lower is better).

4 Results

Plain baseline frontier

model	total FLOPs	BPB
d=10	0.49×10^{18}	0.8769
d=12	1.17×10^{18}	0.8307
d=14	2.55×10^{18}	0.7967
d=16	5.11×10^{18}	**0.7737**

Gated readers (vs. frontier at equal FLOPs)

reader	BPB	frontier	gap / gate
V=2	0.8665	0.834	+0.033 / 0.40
V=4	0.8570	0.810	+0.047 / 0.35
V=8	0.8431	0.779	+0.064 / 0.39
V=10	0.8376	0.768	+0.070 / 0.40

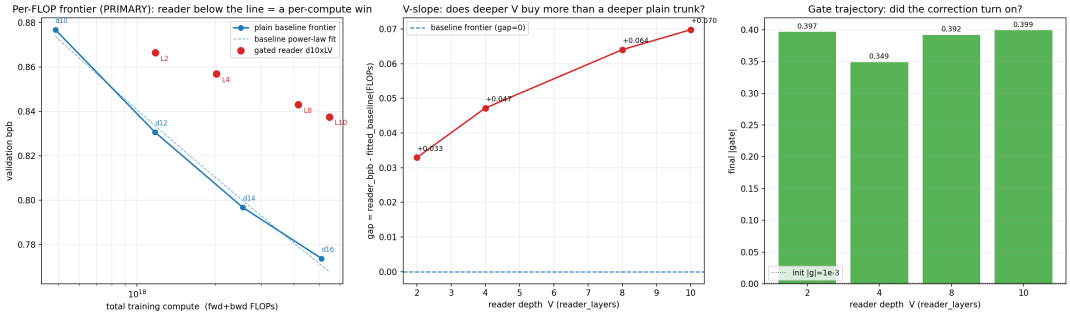

Figure 1: **Left:** the four plain baselines (blue) trace a loss-vs-compute frontier; all four gated readers (red) sit *above* it — a per-compute loss at every depth. **Middle:** the gap to the frontier *grows* with reader depth. **Right:** the gate is firmly on ($|g| \approx 0.35$–0.40, from a 10^{-3} init) in every run — the reader genuinely turned on.

Three things stand out:

1. **Every reader loses per-FLOP**, and the gap *widens* with V (+0.033 → +0.070 BPB). At $\sim 5 \times 10^{18}$ FLOPs, a plain d=16 scores 0.774 while the deepest reader (V=10) reaches only 0.838 — the baseline wins by 0.064. Plain depth scales ∼3× better per FLOP.
2. **It is not a dead component.** The gate turned on hard in every run, so the reader *did* learn and contribute. This is a real negative result, not an artifact of V doing nothing.
3. **The "equal-data" win is a mirage.** The readers do beat the d=10 anchor (0.877) token-for-token — but only because they are bigger models fed more tokens. Matched on *compute*, they lose.

5 Conclusion

Even in its most favorable form — additive, gated, and free to learn to turn itself on — the depth-reader is a **worse use of compute than simply making the trunk deeper**, and *increasingly so* with reader depth. The takeaway is blunt: spend the FLOPs on depth, not on re-reading the ladder.